PROJECT AND TEAM INFORMATION

Project Title

SmartFile: C++ Based File Compression and Decompression Using Huffman Coding

Student / Team Information

Team Name: *Team #*	*BitMasters*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Negi , Rishi – 2510380076* *2510380076@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Bansal , Vardan – 2510372517* *2510372517@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Rautela , Priyansh - 2510370501* *2510370501@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Saksham , Pragya - 2510370412* *2510370412@geu.ac.in* 

PROPOSAL DESCRIPTION

Motivation

Today, academic work, programming files, notes, and other documents are increasingly kept in digital form. As the number of files grows, available storage can be consumed quickly. Large files can also take longer to transfer through email or messaging platforms and may require more data. File compression provides a practical way to reduce storage and transfer requirements while preserving the original information.
This project was selected as an opportunity to apply DSA concepts in a practical C++ application. The implementation uses binary trees and a priority queue together with Huffman Coding, a greedy compression technique. In this method, frequently occurring bytes receive shorter codes, while less frequent bytes receive longer codes. C++ and Object-Oriented Programming are used to divide the application into manageable classes and modules. The project also provides hands-on practice with dynamic memory management, tree structures, and binary file operations.

State of the Art / Current solution

Several established applications, including WinRAR, ZIP, and 7-Zip, provide file compression and decompression. Depending on the format and implementation, such tools may combine techniques such as LZ77 and Huffman Coding to obtain compact representations of data. Examples of widely used compression techniques include:

- ***Huffman Coding:*** *Builds variable-length codes so that frequently occurring symbols generally use fewer bits than symbols that occur less often.*
- ***Run-Length Encoding (RLE):*** *Represents consecutive repeated values using the value together with its repetition count*
- ***Lempel-Ziv (LZ):*** *Detects repeated sequences and represents later occurrences through references rather than storing the complete sequence again.*

Although commercial compression utilities are powerful, their complete implementation can be difficult for beginners to study. SmartFile is therefore designed as a simpler educational implementation that demonstrates how compression works internally. It connects practical file processing with data structures such as Huffman trees and priority queues. Compression is not always beneficial for small inputs. Header and metadata information can add enough overhead to make the generated file larger than the source. SmartFile therefore compares the resulting size with the original and uses a RAW mode when compression would not provide a size reduction.

Project Goals and Milestones

The main goal of this project is to develop a file compression and decompression system using Huffman Coding while applying the data structures and programming concepts learned during our course. To achieve this, we have set the following milestones:

- *Open the selected input in binary mode and process it byte-by-byte so that the same mechanism can handle different file contents.*
- *Build a frequency table that records how often each byte occurs in the input.*
- *Construct the Huffman Tree with a Priority Queue implemented as a min-heap.*
- *Traverse the resulting tree and assign a sequence of 0s and 1s to every byte represented in the tree.*
- *Implement BitWriter and BitReader classes so individual bits can be packed and recovered instead of being stored as ordinary character text.*
- *Define a custom .sfile binary format whose header contains the metadata needed for decompression, including original size and frequency information.*
- *During decompression, read the saved header and use its frequency data to reconstruct the same Huffman Tree.*
- *Read the encoded bit stream, decode it through the tree, and reproduce the source file without changing its byte content.*
- *Use OOP by separating responsibilities into classes such as HuffmanNode, BitWriter, BitReader, and Compressor.*
- *Show useful results after processing, such as source size, output size, and the amount of storage saved.*

By completing these milestones, we aim to gain practical experience in implementing algorithms, working with file handling, and understanding how data compression techniques are used in real-world applications.

Project Approach

The application is being implemented in C++ with Object-Oriented Programming as the main design approach.

How the Program Works

1.Reading the Input

The selected file is opened in binary mode. Each byte is read and its occurrence is recorded in a frequency table.

2.Building the Huffman Tree:

Only bytes that occur in the input are inserted into the min-heap. The two least-frequent nodes are removed, joined under a new parent node, and the parent is inserted again. Repeating this operation produces the root of the Huffman Tree.

3.Creating Codes:

A left branch represents 0 and a right branch represents 1. Traversing from the root to each leaf produces the code assigned to that byte. Before writing the compressed result, the program also considers the header overhead and determines whether the final output will actually be smaller.

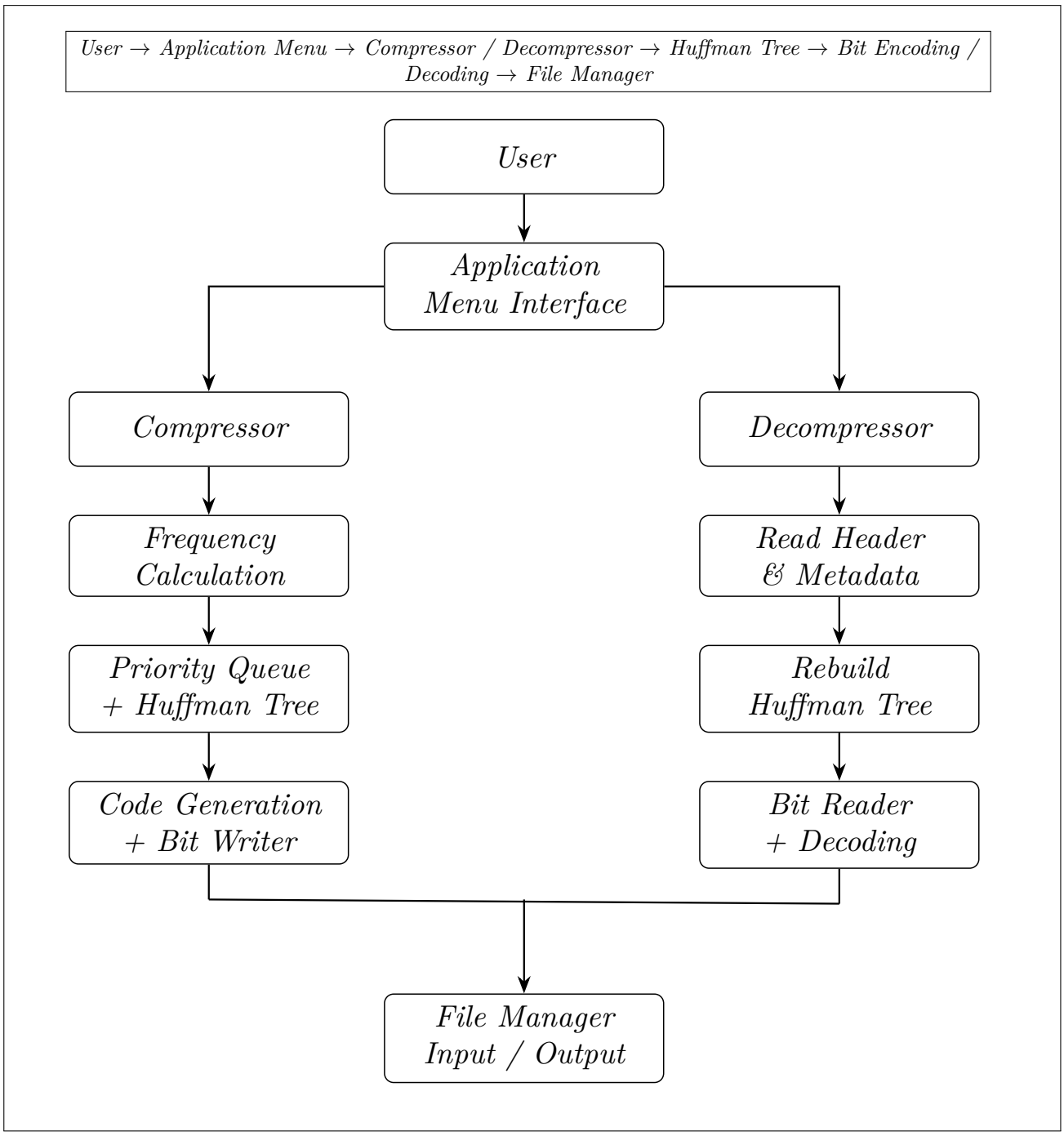
4.Compressing the File

- *When the Huffman result is smaller, SmartFile writes the required metadata and packs the encoded bits into the output through BitWriter.*
- *When compression would increase the total file size, the program selects RAW mode and stores the source bytes without Huffman encoding.*

5.Restoring the File

The decompressor first reads the custom header and identifies the selected storage mode. For RAW data, the stored bytes are copied back directly. For Huffman data, the saved frequency information is used to recreate the tree; BitReader then supplies the encoded bits, which are decoded by walking the tree until the original bytes are recovered..

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The completed deliverable will be a C++ application capable of both compressing files and recovering them through the corresponding decompression process. ***Planned Deliverables:***

- *Well-structured C++ implementation separated into header (.h) and source (.cpp) files.*
- *Implementation of the Huffman Tree together with a min-heap based Priority Queue.*
- *Bit-level input and output functionality for compact storage of encoded data.*
- *A custom binary .sfile container with a metadata header.*
- *RAW storage as an automatic alternative when Huffman compression does not produce a smaller result.*
- *A results display containing source size, compressed size, and storage-saving percentage.*
- *Verification tests demonstrating that decompression reproduces the input file byte-for-byte.*

By completing this project, we will better understand how the concepts learned in Data Structures and Algorithms can be applied to solve real-world problems such as file compression and storage optimization.

Assumptions

The following assumptions have been made while developing this project:

- *The user provides a valid input file, and the system has permission to read and create files.*
- *Huffman Coding is a lossless , so the original data can be recovered completely after decompression .*
- *The compressed file is assumed to be generated by our program and not damaged or modified.*
- *Compression performance depends on the file type. Text and repetitive files may achieve better compression, while already compressed files such as .zip and .png may show little reduction.*
- *Sufficient storage space is available for compressed and decompressed files.*

These conditions define the basic requirements for the system to work correctly and produce accurate results

References

- *Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes.*
- *Weiss, Mark Allen. Data Structures and Algorithms in C++.*
- *C++ Reference Guides , including cppreference.com , for std :: priority_queue and file handling functions.*